Mutex ինչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացatrum e mutex-y՝ inch e, inchpes e lucum race condition-y, lock/unlock, lock_guard, unique_lock, orinaknerov ev interview harcerov.

---

1. Ի՞նչ է Mutex-ը

Mutex = **MUT**ual **EX**clusion. Sinxronizacian mekanizm, vor apahovum e, vor **miayn mek thread** miangamic mutq uni shared resource-in (critical section)։

```
Mutex = "banali" -> miayn mek thread karox e "kaxel" ayn miangamic
  lock()   -> vercnum banalin (ete zbaxvac e, spasum)
  unlock() -> veradarcnum banalin
```

```cpp
#include <mutex>
std::mutex mtx;
int counter = 0;

void increment() {
    mtx.lock();       // critical section sksvum
    counter++;
    mtx.unlock();     // critical section avartvum
}
```

---

2. Xndiry arańc Mutex (Race Condition)

```
int counter = 0;
void increment() {
    for (int i = 0; i < 100000; i++)
        counter++;    // 3 qayl: read, +1, write -> thread-ner xarńvum
}
// 2 thread -> counter != 200000 (race condition)
```

`counter++` -y **atomic che** (read-modify-write)。Erku thread miasin -> arjeqner korchen.

---

3. Lucumy Mutex-ov

```
std::mutex mtx;
int counter = 0;

void increment() {
    for (int i = 0; i < 100000; i++) {
        mtx.lock();
        counter++;      // miayn mek thread miangamic
        mtx.unlock();
    }
}
// hima counter == 200000 (chisht)
```

Mutex-y apahovum e, vor `counter++` -y katarvi anbajanelov (mek thread miangamic).

---

4. lock_guard (RAII, NAXENTRELI)

Dzeŕqov `lock()` / `unlock()` -y vtangavor e (exception -> unlock chi hasni -> deadlock).
`lock_guard` -y RAII e.

```cpp
std::mutex mtx;
int counter = 0;

void increment() {
    std::lock_guard<std::mutex> lock(mtx);   // ctor -> lock
    counter++;
}   // dtor -> unlock (avtomat, nuynisk exception-i depqum)
```

`lock_guard`-y. lock ctor-um, unlock dtor-um -> chka "moŕacɯ unlock".

---

5. unique_lock (aveli chkun)

```cpp
std::unique_lock<std::mutex> lock(mtx);
// ...
lock.unlock();   // dzeŕqov unlock karox
lock.lock();     // noric lock
// condition_variable-i het ogtagorcvum
```

```
lock_guard   -> paŕz, lock ctor / unlock dtor (chi bacvum)
unique_lock  -> chkun (dzeŕqov lock/unlock, defer, condition_variable)
```

---

6. Mutex-i tesaknery

```cpp
std::mutex              // sovorakan
std::recursive_mutex    // nuyn thread-y karox e mi qani angam lock anel
std::timed_mutex        // try_lock_for (timeout-ov)
std::shared_mutex       // shared (read) + exclusive (write) -- C++17
```

---

7. Karevor kanonner

```
- Misht lock_guard/unique_lock (ne dzeŕqov lock/unlock)
- Kritik seksian pahi POQR (aveli qich blocking)
- Nuyn kargov lock ani mi qani mutex (deadlock-ic xusapel)
- Chka lock-i mej erkar operacia (I/O)
```

---

8. Interview harcer & patasxanner

Mutex inch e?

> *Mutual Exclusion -- sinxronizacian mekanizm, miayn mek thread miangamic mutq shared resource-in.*

Inch problem e lucum?

> *Race condition -- shared data-i anvtang mutq.*

lock() / unlock()?

> *lock vercnum banalin (spasum ete zbaxvac); unlock veradarcnum.*

lock_guard inchu naxentreli?

> *RAII -- lock ctor / unlock dtor; exception-safe, chka moŕacum.*

lock_guard vs unique_lock?

> *lock_guard paŕz (chi bacvum); unique_lock chkun (dzeŕqov lock/unlock, condition_variable).*

recursive_mutex inch e?

> *Nuyn thread-y karox e mi qani angam lock anel (sovorakan mutex-y -> deadlock).*

Mutex-i vtangy?

> *Deadlock (ete sxal ogtagorces), performance (blocking).*

Cheat Sheet

```
MUTEX = Mutual Exclusion -> miayn mek thread miangamic critical section-um

  lock() / unlock()  -> dzeŕqov (VTANGAVOR)
  lock_guard         -> RAII (lock ctor/unlock dtor) NAXENTRELI
  unique_lock        -> chkun (dzeŕqov lock/unlock, condition_variable)

TESAKNER: mutex | recursive_mutex | timed_mutex | shared_mutex (C++17)

LUCUM: race condition (shared data anvtang mutq)
KANON: lock_guard ogtagorci | kritik seksian poqr | nuyn kargov lock (deadlock xusapel)
```